MATH2270/MATH2237/MATH2404 Assignment 3

Name
Dhyey U Vyas

Student number
S4097968

Assessment declaration checklist
Please carefully read the statements below and check each box if you agree with the declaration. If you do not check all boxes, your assignment will not be marked. If you make a false declaration on any of these points, you may be investigated for academic misconduct. Students found to have breached academic integrity may receive official warnings and/or serious academic penalties. Please read more about academic integrity here. If you are unsure about any of these points or feel your assessment might breach academic integrity, please contact your course coordinator for support. It is important that you DO NOT submit any assessment until you can complete the declaration truthfully.

By checking the boxes below, I declare the following:

- ☒ I have not impersonated, or allowed myself to be impersonated by, any person for the purposes of this assessment
- ☒ This assessment is my original work and no part of it has been copied from any other source except where due acknowledgement is made.
- ☒ No part of this assessment has been written for me by any other person except where such collaboration has been authorised by the lecturer/teacher concerned.
- ☒ I have not used generative "AI" tools for the purposes of this assessment.
- ☒ Where this work is being submitted for individual assessment, I declare that it is my original work and that no part has been contributed by, produced by or in conjunction with another student.
- ☒ I give permission for my assessment response to be reproduced, communicated, compared and archived for the purposes of detecting plagiarism.
- ☒ I give permission for a copy of my assessment to be retained by the university for review and comparison, including review by external examiners.

I understand that:

- ☒ Plagiarism is the presentation of the work, idea or creation of another person or machine as though it is your own. It is a form of cheating and is a very serious academic offence that may lead to exclusion from the University. Plagiarised material can be drawn from, and presented in, written, graphic and visual form, including electronic data and oral presentations. Plagiarism occurs when the origin of the material used is not appropriately cited.
- ☒ Plagiarism includes the act of assisting or allowing another person to plagiarise or to copy my work.

I agree and acknowledge that:

- ☒ I have read and understood the Declaration and Statement of Authorship above.

☒ If I do not agree to the Declaration and Statement of Authorship in this context and all boxes are not checked, the assessment outcome is not valid for assessment purposes and will not be included in my final result for this course.

Assignment URL

https://b33wia-dhyey0u-vyas.shinyapps.io/f1-starting-position-dashboard/

References

Rao, R. (2024). *Formula 1 World Championship Dataset (1950–2023)* [Data set]. Kaggle. https://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020

Chang, W., Cheng, J., Allaire, J. J., Sievert, C., Schloerke, B., Xie, Y., & McPherson, J. (2023). *Shiny: Web application framework for R* (Version 1.8.0) [Computer software]. https://CRAN.R-project.org/package=shiny

Assignment Code

```
# Assignment 3 – Storytelling with Open Data

# Dashboard Title: F1 Starting Position Analysis

# Author: Dhyey U Vyas

# Student ID: S4097968



# Summary:

# This Shiny dashboard explores the impact of starting position on Formula 1 race results

# using data from the Formula 1 World Championship (1950–2023). It includes:

# 1. Pole position win rate analysis across eras,

# 2. Grid vs finish position scatter insights,

# 3. Most dramatic comeback drives in F1 history.



# Data source: Rao, R. (2024). Kaggle F1 Dataset (1950–2023)



# Load libraries

library(shiny)
```

```
library(shinydashboard)
library(DT)
library(plotly)
library(dplyr)
library(readr)
library(janitor)
library(ggplot2)

# TODO: Make sure CSV files are in the same directory when deploying!
# Data prep function - keeping it simple for now
prepare_f1_data <- function() {
  # Read the CSV files with error handling
  tryCatch({
    qualifying <- read_csv("qualifying.csv") %>% clean_names()
    results <- read_csv("results.csv") %>% clean_names()
    races <- read_csv("races.csv") %>% clean_names()
    drivers <- read_csv("drivers.csv") %>% clean_names()
    constructors <- read_csv("constructors.csv") %>% clean_names()
  }, error = function(e) {
    stop("Error reading CSV files. Make sure all CSV files are in the working directory: ", e$message)
  })

  # Join everything together - this took me ages to figure out!
  # Had to debug the constructor_id issue by printing column names
  combined_data <- qualifying %>%
    inner_join(results, by = c("race_id", "driver_id")) %>%
    inner_join(races, by = "race_id") %>%
    inner_join(drivers, by = "driver_id")
```

```
# Handle the constructor join - column names were confusing
if("constructor_id.x" %in% names(combined_data)) {
  combined_data <- combined_data %>%
    inner_join(constructors, by = c("constructor_id.x" = "constructor_id"))
} else if("constructor_id.y" %in% names(combined_data)) {
  combined_data <- combined_data %>%
    inner_join(constructors, by = c("constructor_id.y" = "constructor_id"))
} else {
  combined_data <- combined_data %>%
    inner_join(constructors, by = "constructor_id")
}

# Clean up the data and create useful columns
final_data <- combined_data %>%
  filter(!is.na(position.x), !is.na(position.y)) %>%
  mutate(
    grid_pos = as.numeric(position.x),
    finish_pos = as.numeric(ifelse(position.y == "\\N", NA, position.y)),
    positions_gained = grid_pos - finish_pos,  # positive = gained positions
    driver_full_name = paste(forename, surname),
    team = name.y,  # Constructor name
    grand_prix = name.x,  # Race name
    won_from_pole = ifelse(grid_pos == 1 & finish_pos == 1, TRUE, FALSE),
    started_pole = ifelse(grid_pos == 1, TRUE, FALSE),
    got_podium = ifelse(finish_pos <= 3 & !is.na(finish_pos), TRUE, FALSE),
    # Group years into eras for analysis
    f1_era = case_when(
      year <= 1970 ~ "Early Years (1950-1970)",
      year <= 1990 ~ "Turbo Era (1971-1990)",
```

```
      year <= 2010 ~ "Modern Era (1991-2010)",
      TRUE ~ "Hybrid Era (2011+)"
    )
  ) %>%
  # Remove invalid data - be more careful with filtering
  filter(
    grid_pos > 0,
    !is.na(finish_pos),
    finish_pos > 0,
    !is.na(positions_gained),
    !is.na(driver_full_name),
    !is.na(team)
  )

 return(final_data)
}

# Load the data with error handling
tryCatch({
 f1_data <- prepare_f1_data()
 cat("Data loaded successfully. Rows:", nrow(f1_data), "\n")
}, error = function(e) {
 stop("Failed to prepare F1 data: ", e$message)
})

# UI Definition
ui <- dashboardPage(
 dashboardHeader(title = "F1 Starting Position Analysis"),
```

```
dashboardSidebar(
  sidebarMenu(
    menuItem("Pole Position Analysis", tabName = "pole_tab", icon = icon("flag-checkered")),
    menuItem("Grid vs Finish", tabName = "scatter_tab", icon = icon("chart-line")),
    menuItem("Best Comebacks", tabName = "comeback_tab", icon = icon("trophy"))
  )
),

dashboardBody(
  # Custom CSS to make it look nicer
  tags$head(
    tags$style(HTML("
      .content-wrapper { background-color: #f4f4f4; }
      .box { margin-bottom: 20px; }
      h4 { color: #333; margin-top: 0; }
    "))
  ),

  tabItems(
    # First tab - Pole position analysis
    tabItem(
      tabName = "pole_tab",

      # Introduction box
      fluidRow(
        box(
          title = "Does Pole Position Really Matter in F1?",
          status = "primary",
          solidHeader = TRUE,
```

```
    width = 12,
    p("This dashboard explores whether starting from pole position (P1) actually leads to race wins in Formula 1."),
    p("Using F1 data from 1950-2023, we can see how the pole position advantage has changed over different eras of the sport.")
   )
  ),

  fluidRow(
   # Main chart
   box(
    title = "Pole Position Win Rate Over Time",
    status = "primary",
    solidHeader = TRUE,
    width = 8,
    selectInput("era_select", "Choose Era:",
          choices = c("All Eras", sort(unique(f1_data$f1_era))),
          selected = "All Eras"),
    plotlyOutput("pole_chart", height = "350px")
   ),

   # Summary stats
   box(
    title = "Key Statistics",
    status = "info",
    solidHeader = TRUE,
    width = 4,
    h4("Pole Position Facts:"),
    p("• Overall, pole sitters win about 40% of races"),
```

```
      p("• This varies significantly between different F1 eras"),
      p("• Modern F1 shows declining pole advantage due to closer competition"),
      br(),
      p(strong("Fun fact:"), "Some drivers are much better at converting pole to wins than others!"),
      br(),
      tableOutput("quick_stats")
    )
  ),

  # Summary table
  fluidRow(
    box(
      title = "Win Rates by Era",
      status = "success",
      solidHeader = TRUE,
      width = 12,
      DT::dataTableOutput("era_table")
    )
  )
),

# Second tab - Scatter plot analysis
tabItem(
  tabName = "scatter_tab",
  fluidRow(
    box(
      title = "Starting Grid Position vs Final Result",
      status = "primary",
      solidHeader = TRUE,
```

```
        width = 9,
        p("Each point represents one driver's performance in a race. Points below the diagonal line show position gains."),
        selectInput("year_filter", "Select Time Period:",
                    choices = list(
                      "All Years (1950-2023)" = "all",
                      "Recent (2020-2023)" = "recent",
                      "2010s" = "2010s",
                      "2000s" = "2000s",
                      "1990s" = "1990s",
                      "Before 1990" = "early"
                    )),
        plotlyOutput("position_scatter", height = "400px")
      ),

      box(
        title = "What This Shows",
        status = "warning",
        solidHeader = TRUE,
        width = 3,
        h4("Reading the Chart:"),
        p("• Front row (P1-P2) gives best chance of winning"),
        p("• Points below diagonal = positions gained"),
        p("• Points above diagonal = positions lost"),
        p("• Orange dots = podium finishes"),
        br(),
        p(strong("Insight:"), "Starting in the top 10 gives you the best shot at points, but remarkable drives can come from anywhere on the grid!")
      )
```

```
  )
),

# Third tab - Comeback drives
tabItem(
  tabName = "comeback_tab",
  fluidRow(
    box(
      title = "Most Impressive Comeback Drives",
      status = "success",
      solidHeader = TRUE,
      width = 8,
      p("These are the drives where drivers gained the most positions from start to finish:"),
      selectInput("comeback_type", "Show:",
                  choices = list(
                    "All Time Greatest" = "all_time",
                    "Last 10 Years" = "recent_years",
                    "Specific Team" = "by_team"
                  )),
      conditionalPanel(
        condition = "input.comeback_type == 'by_team'",
        selectInput("team_select", "Choose Team:", choices = NULL)
      ),
      plotOutput("comeback_plot", height = "350px")
    ),

    box(
      title = "Amazing Recovery",
      status = "info",
```

```
        solidHeader = TRUE,
        width = 4,
        h4("Featured Comeback:"),
        verbatimTextOutput("featured_comeback"),
        br(),
        h4("What Makes Great Comebacks?"),
        p("• Rain and changing conditions"),
        p("• Smart pit stop strategy"),
        p("• Safety car timing"),
        p("• Pure driving skill"),
        p("• Reliability when others fail"),
        br(),
        em("These drives show why we love F1 - anything can happen on race day!")
      )
    )
  )
),

# Footer
fluidRow(
  box(
    width = 12,
    p(strong("Data Source:"), "Rao, R. (2024). Formula 1 World Championship Dataset (1950–2023). Kaggle.",
      a("https://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020",
        href="https://www.kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020")),
    p(em("Created for Data Visualization Assignment 3"))
  )
)
```

```
  )
)

# Server logic
server <- function(input, output, session) {

  # Update team choices for comeback analysis
  observe({
    req(f1_data) # Ensure data is loaded
    team_options <- sort(unique(f1_data$team))
    updateSelectInput(session, "team_select",
                      choices = team_options)
  })

  # Pole position chart
  output$pole_chart <- renderPlotly({
    req(f1_data) # Ensure data is loaded

    # Filter based on era selection
    data_to_use <- if(input$era_select == "All Eras") {
      f1_data
    } else {
      f1_data %>% filter(f1_era == input$era_select)
    }

    # Validate filtered data
    if(nrow(data_to_use) == 0) {
      return(plotly_empty() %>% layout(title = "No data available for selected era"))
    }
```

```
# Calculate yearly win rates
yearly_stats <- data_to_use %>%
  group_by(year) %>%
  summarise(
    poles = sum(started_pole, na.rm = TRUE),
    pole_wins = sum(won_from_pole, na.rm = TRUE),
    win_percentage = ifelse(poles > 0, (pole_wins/poles) * 100, 0),
    .groups = 'drop'
  ) %>%
  filter(poles > 0)

if(nrow(yearly_stats) == 0) {
  return(plotly_empty() %>% layout(title = "No pole position data available"))
}

# Create the plot
p <- ggplot(yearly_stats, aes(x = year, y = win_percentage)) +
  geom_col(fill = "#e74c3c", alpha = 0.7) +
  geom_smooth(se = FALSE, color = "#2c3e50", method = "loess") +
  labs(
    x = "Year",
    y = "Win Rate (%)",
    title = "How Often Does Pole Position Lead to Victory?"
  ) +
  theme_minimal() +
  ylim(0, 100)

ggplotly(p) %>% config(displayModeBar = FALSE)
```

```
})

# Quick stats table
output$quick_stats <- renderTable({
  req(f1_data)

  overall_stats <- f1_data %>%
    summarise(
      `Total Poles` = sum(started_pole, na.rm = TRUE),
      `Pole Wins` = sum(won_from_pole, na.rm = TRUE),
      `Success Rate` = paste0(round((sum(won_from_pole, na.rm = TRUE)/sum(started_pole, na.rm = TRUE))*100, 1), "%")
    )

  data.frame(
    Metric = c("All-Time Pole Win Rate", "Total Pole Positions", "Races Won from Pole"),
    Value = c(overall_stats$`Success Rate`, overall_stats$`Total Poles`, overall_stats$`Pole Wins`)
  )
}, bordered = TRUE)

# Era comparison table
output$era_table <- DT::renderDataTable({
  req(f1_data)

  era_stats <- f1_data %>%
    group_by(f1_era) %>%
    summarise(
      `Pole Positions` = sum(started_pole, na.rm = TRUE),
      `Wins from Pole` = sum(won_from_pole, na.rm = TRUE),
```

```
    `Win Rate (%)` = round((`Wins from Pole`/`Pole Positions`) * 100, 1),
    .groups = 'drop'
  ) %>%
  filter(`Pole Positions` > 0) %>%
  arrange(desc(`Win Rate (%)`))

 DT::datatable(era_stats, options = list(pageLength = 5, dom = 't'), rownames = FALSE)
})

# Scatter plot
output$position_scatter <- renderPlotly({
 req(f1_data)

 # Filter by time period
 plot_data <- switch(input$year_filter,
                     "all" = f1_data,
                     "recent" = f1_data %>% filter(year >= 2020),
                     "2010s" = f1_data %>% filter(year >= 2010 & year < 2020),
                     "2000s" = f1_data %>% filter(year >= 2000 & year < 2010),
                     "1990s" = f1_data %>% filter(year >= 1990 & year < 2000),
                     "early" = f1_data %>% filter(year < 1990),
                     f1_data
 )

 if(nrow(plot_data) == 0) {
  return(plotly_empty() %>% layout(title = "No data available for selected period"))
 }

 # Sample if too many points
```

```
  if(nrow(plot_data) > 3000) {
    plot_data <- sample_n(plot_data, 3000)
  }

  # Create scatter plot
  p <- ggplot(plot_data, aes(x = grid_pos, y = finish_pos, color = got_podium,
                             text = paste("Driver:", driver_full_name,
                                          "<br>Team:", team,
                                          "<br>Year:", year,
                                          "<br>Started P", grid_pos, "-> Finished P", finish_pos,
                                          "<br>Gained:", positions_gained, "positions"))) +
    geom_point(alpha = 0.5, size = 1) +
    geom_abline(slope = 1, intercept = 0, linetype = "dashed", alpha = 0.5) +
    scale_color_manual(values = c("FALSE" = "gray60", "TRUE" = "orange"),
                       name = "", labels = c("Other", "Podium")) +
    labs(x = "Starting Position", y = "Finishing Position",
         title = "Grid vs Finish Position") +
    theme_minimal() +
    xlim(1, 24) + ylim(1, 24)

  ggplotly(p, tooltip = "text") %>% config(displayModeBar = FALSE)
})

# Comeback chart
output$comeback_plot <- renderPlot({
  req(f1_data)

  comeback_data <- switch(input$comeback_type,
                          "all_time" = f1_data,
```

```
                "recent_years" = f1_data %>% filter(year >= 2014),
                "by_team" = {
                  if(!is.null(input$team_select) && input$team_select != "") {
                    f1_data %>% filter(team == input$team_select)
                  } else {
                    f1_data %>% slice(0) # Empty data frame
                  }
                },
                f1_data
)

# Get biggest comebacks - filter out any negative or zero gains
# Group by driver to get their best comeback only
best_comebacks <- comeback_data %>%
  filter(positions_gained > 5, !is.na(positions_gained)) %>%  # Only significant gains
  group_by(driver_full_name, team) %>%
  slice_max(positions_gained, n = 1, with_ties = FALSE) %>%  # Get best comeback per driver
  ungroup() %>%
  arrange(desc(positions_gained)) %>%
  head(12) %>%
  mutate(
    driver_team = paste0(driver_full_name, " (", team, ")"),
    gain_label = paste0("+", positions_gained)
  )

# Check if we have data
if(nrow(best_comebacks) == 0) {
  # Create empty plot with message
  ggplot() +
```

```
      annotate("text", x = 0.5, y = 0.5, label = "No significant comebacks found", size = 6) +
      theme_void()
  } else {
    ggplot(best_comebacks, aes(x = reorder(driver_team, positions_gained),
                               y = positions_gained)) +
      geom_col(fill = "forestgreen", alpha = 0.8, width = 0.7) +
      geom_text(aes(label = gain_label), hjust = -0.1, size = 3, color = "black") +
      coord_flip() +
      labs(
        x = "Driver (Constructor)",
        y = "Positions Gained",
        title = "Greatest Comeback Drives"
      ) +
      theme_minimal() +
      theme(
        axis.text.y = element_text(size = 9),
        axis.text.x = element_text(size = 10),
        axis.title = element_text(size = 11, face = "bold"),
        plot.title = element_text(size = 12, face = "bold"),
        panel.grid.major.y = element_blank(),
        panel.grid.minor = element_blank()
      ) +
      scale_y_continuous(expand = expansion(mult = c(0, 0.15))) # Give space for labels
  }
})

# Featured comeback story
output$featured_comeback <- renderText({
  req(f1_data)
```

```
  comeback_data <- switch(input$comeback_type,
                          "all_time" = f1_data,
                          "recent_years" = f1_data %>% filter(year >= 2014),
                          "by_team" = {
                            if(!is.null(input$team_select) && input$team_select != "") {
                              f1_data %>% filter(team == input$team_select)
                            } else {
                              f1_data %>% slice(0) # Empty data frame
                            }
                          },
                          f1_data
  )

  best_drive <- comeback_data %>%
    filter(positions_gained > 0, !is.na(positions_gained)) %>%
    arrange(desc(positions_gained)) %>%
    slice(1)

  if(nrow(best_drive) > 0) {
    paste0(best_drive$driver_full_name, " at the ", best_drive$year, " ",
           best_drive$grand_prix, " gained ", best_drive$positions_gained,
           " positions!\n\nStarted P", best_drive$grid_pos,
           " and finished P", best_drive$finish_pos, ".")
  } else {
    "No major comebacks found in this selection."
  }
 })
}
```

```
# Run the app

shinyApp(ui = ui, server = server)
```